

lab2-Java Parallel Sorting

Dawa Arkhang and Zhengan Chen

September 2026

1 Sequential Sort

We decided to implement quicksort and used the classic method of recursively choosing a pivot point, which in our case was the middle element, swapping its place with the last element in the array. The next step is then to iterate through the array, skipping the last element(our pivot element) and proceeding to swap the elements smaller or equal to the pivot to the left of it and those larger than the pivot to the right of it. Finally, the pivot is swapped into its definitive sorted position between the two partitions, and the algorithm recursively sorts the left and right subarrays.

2 Amdahl's law

The classical formulation of Amdahl's law cannot be applied naively to recursive Quicksort because the concurrency is constrained by the width of the recursion tree at each stage. To model the gradual ramp-up across levels up to 16 threads, we divide the parallelization portion p into five equal phases. Furthermore, the $\min()$ function is used in the denominators because the maximum number of active threads at any given level is strictly limited by the number of independent subarrays available to sort.

$$T(s) = (1 - p) + \frac{\frac{p}{5}}{1} + \frac{\frac{p}{5}}{\min(2, s)} + \frac{\frac{p}{5}}{\min(4, s)} + \frac{\frac{p}{5}}{\min(8, s)} + \frac{\frac{p}{5}}{\min(16, s)} \quad (1)$$

$T(s)$ here stands for the total time that the program will take. The theoretical speedup $S(s)$ is consequently given by:

$$S(s) = \frac{1}{T(s)} = \frac{1}{(1 - p) + \frac{\frac{p}{5}}{1} + \frac{\frac{p}{5}}{\min(2, s)} + \frac{\frac{p}{5}}{\min(4, s)} + \frac{\frac{p}{5}}{\min(8, s)} + \frac{\frac{p}{5}}{\min(16, s)}} \quad (2)$$

We can therefore see the result of this in Figure 3. The reason to why our version of Amdhals law is better than the general one is due to the fact that it accounts for the non-uniform, hierarchical parallelism in Quicksort. In the

general version, it assumes that all parallel work is evenly distributed across the threads at the same time. It is noteworthy to know that this formula works in the assumption that the array that we are using quicksort on will have enough elements to create 16 independent subarrays.

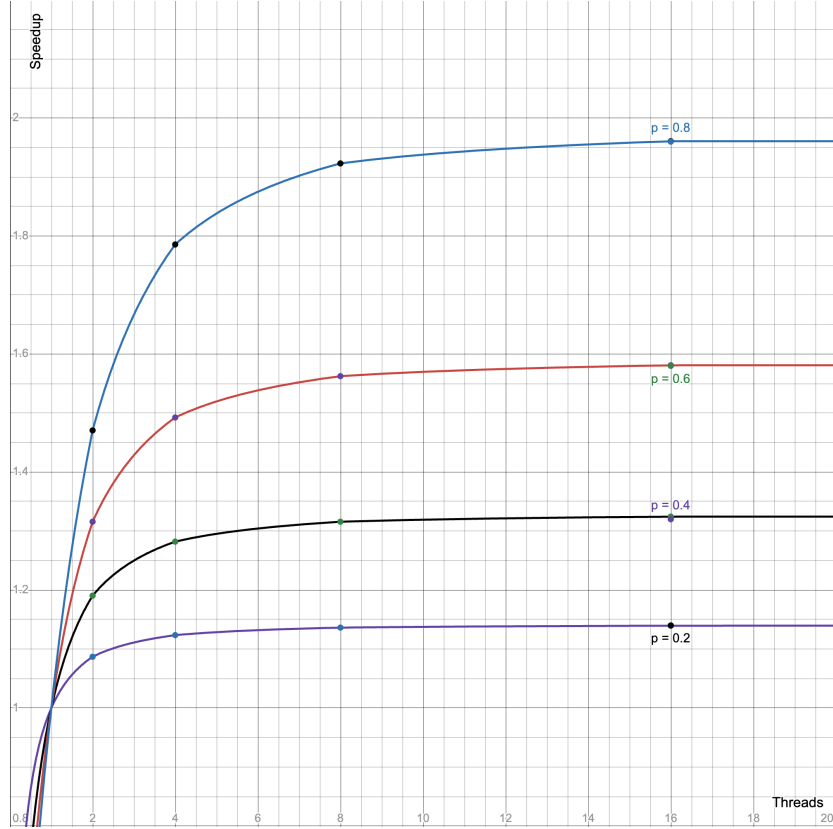


Figure 1: Speedup given by the custom Amdahl's law for recursive Quicksort across thread counts 2, 4, 8, and 16.

3 ExecutorService

Quicksort uses divide and conquer with recursion. At each step, the array is divided into a left part and a right part, which can be processed independently by different threads. However, we only need one new thread, because the thread from the previous level can process one part while the new thread processes the other part. The important thing is that we need to wait for both child nodes to finish before going back to the parent level. Otherwise, the result will be incorrect.

With ExecutorService, we use `submit()` to execute a task and `get()` to wait

for that task to finish. The problem is that while a thread is waiting, it cannot execute another task. Since the submitted task continues to recursively create more tasks, this can cause a situation where the number of threads allocated at the beginning is not enough and the program may never terminate.

Our solution is to determine how deep the recursion tree should go based on the number of available threads. Up to this depth, the recursive calls can run in parallel. After reaching this depth, the algorithm switches back to sequential execution.

Having the number of threads as 2^n , would be ideal for this approach. However, if the number is not a power of two, such as 96, some threads may not have a task to execute. Fully solving this problem might require using locks, but we did not implement that

4 ForkJoinPool and RecursiveAction

We start by creating a forkjoinpool with the amount of threads that we want. The next step is to use invoke on the forkjoinpool as it will then blocks calling threads until the entire computation including all subtasks spawned via fork() finishes. As we are using recursive action, the return value will be void. In the main compute method in the worker, we fork on the left task and continue computing the right side on the current thread as that will decrease task scheduling overhead, eliminate thread stalling and preserves CPU cache locality by running directly as a standard method call on the current thread's call stack.

5 ParallelStream and Lambda Functions

Here, we use the ForkJoinPool based on the hints to limit the number of threads that can be used by the parallel stream. During the recursion, we put the quicksort() function calls into a Stream and then make the stream parallel.

In the recursion, quicksort() usually splits into two smaller quicksort tasks, which can then be executed in parallel. Since creating and running tasks in the stream also has some overhead, when the index range becomes very small, there is no need to keep splitting the tasks. Instead, it can be more efficient to continue with normal sequential execution, like a regular quicksort. In our implementation, we set the range to 4096. This value was chosen empirically based on local benchmarking using "java MeasureMain ParallelStream 8 1000000 10 50 1", where a range of 4096 showed good performance compared with the other values we tested.

6 Performance measurements

The results show that increasing the number of threads generally reduces the execution time. ForkJoinPool and ParallelStream perform better than ExecutorService. JavaSort has the lowest execution time. The results also show

that adding more threads does not always improve performance. For example, changes very little between 64 threads and 96 threads.

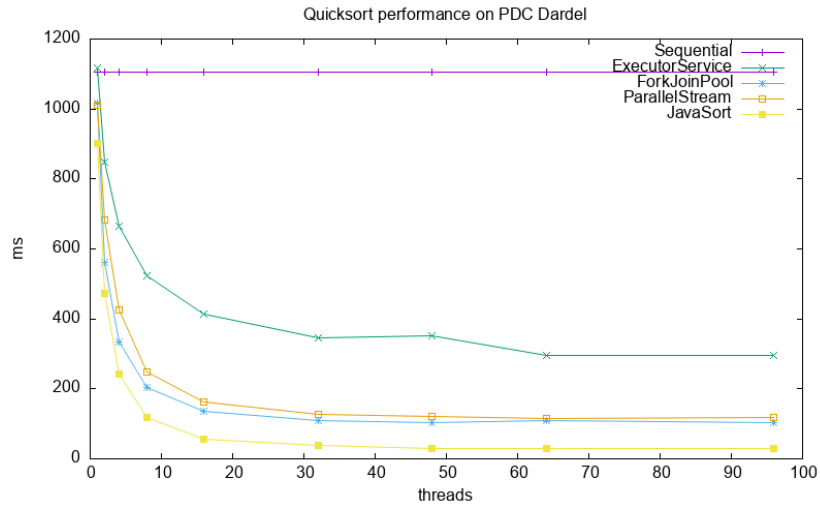


Figure 2: Average execution time vs. number of threads

The speedup generally increases as the number of threads increases. ExecutorService reaches about 3.7x speedup, while ForkJoinPool and ParallelStream reach about 10x and 9x. JavaSort has the highest speedup of about 38.5x with 64 threads.

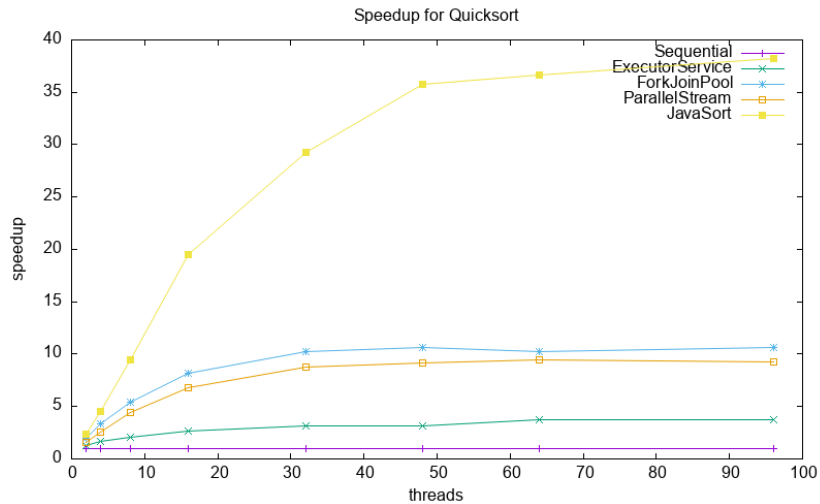


Figure 3: Speedup vs. number of threads

The relatively poor performance of `ExecutorService` was expected. In our `ExecutorService` implementation, only one subarray is submitted to the thread pool at each partition, while the other is processed by the current thread. The parallelism is also limited by the recursion depth. This can leave some threads underutilized. What was more surprising was the large performance gap between `ForkJoinPool/ParallelStream` and `JavaSort`.

It was also expected that performance cannot keep improving as N increases, since the sequential parts of the program limit the maximum speedup. However, the actual speedup was much higher than anticipated.

The easiest implemenataion to write and specifically conceptulize was the `ForkJoinPool` and `RecursiveAction`. This one was specifically easy as the core workings of the code was very easy to understand and follow when writing and there were few to no curve balls that had to be taken into consideration.

The parallelization method that we preferred was the `ForkJoinPool` and `RecursiveAction` implementation due to its very concise syntax and ease of use. Furthermore, its underlying work-stealing algorithm efficiently balances the workload across available threads made it very well suited for quicksort.

Sorting Algorithm	Threads	Average Time (ms)	Standard Deviation (ms)
Sequential	1	4.24	0.09
<code>ForkJoinPool</code>	4	3.01	1.93
<code>ExecutorService</code>	4	2.90	1.04
<code>ParallelStream</code>	4	5.33	2.38
<code>JavaSort</code>	4	1.27	0.12

Table 1: Local validation measurements for an array size of 100,000 elements (5 warm-up rounds, 5 measurement rounds, PRNG seed: 42).